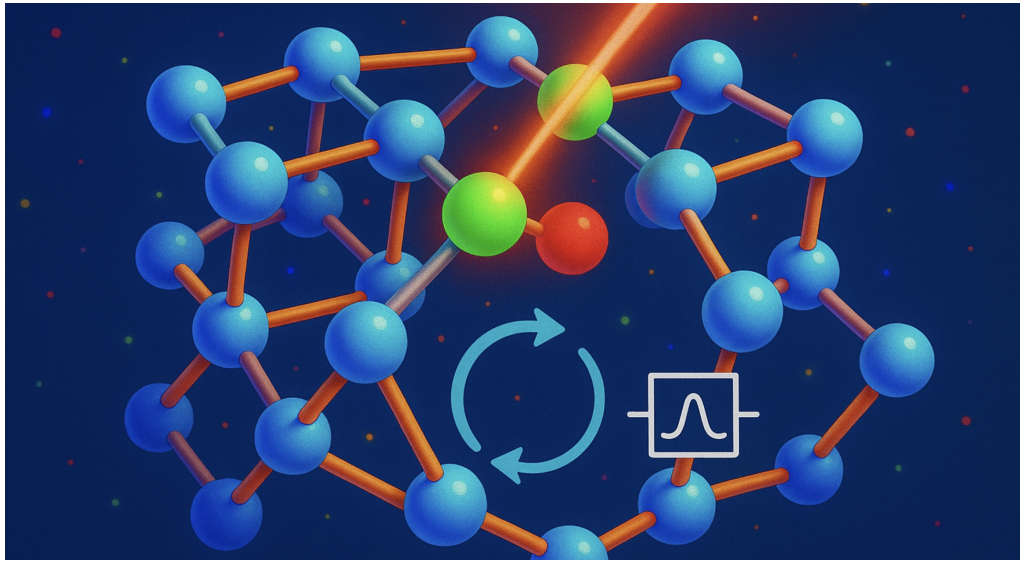
Optimising quantum control systems: application to NV centres

Jean-Baptiste Caillau
Université Côte d'Azur, CNRS, Inria, LJAD

Joint work: **C. Babin, I. Beschastnyi, D. Sugny, D. Tinoco**

JuliaCon 2026
Mainz, August 10-14

NV centres



- Nitrogen-Vacancy centres (color centres in diamond)
- Experimental platform for quantum information processing in lab conditions
- Collaboration with physicists (Nice-Dijon projet funded by CNRS)

  Tinoco, D.; Babin, C.; Beschastnyi, I.; Caillau, J.-B.; Sugny, D. Control of an NV center as a two-qubit system. *IFAC conference* (2026), to appear
- Collaboration Argonne (FACCTS project funded by France-Chicago centre)

  A. Montoison, M. Schanen, ExaModels.jl extension for LinearAlgebra / OptimalControl.jl

Quantum control

- System of n spin-1/2 particles living in $\mathbf{C}^2 \otimes \cdots \otimes \mathbf{C}^2 \simeq \mathbf{C}^{2^n}$
- Gate = unitary operation on $\mathbf{C}^N$, $N = 2^n$: $Q(t) \in SU(N) < GL(\mathbf{C}, N)$
- Schrödinger equation on a compact Lie group of matrices ($\hbar = 1$)

$$\dot{Q} = -iH \cdot Q$$

- Hamiltonian H is an element of the Lie algebra $\mathfrak{su}(N)$ of traceless skew-Hermitian matrices ($H^\dagger = \overline{H}^T = -H$)
- Dimension over $\mathbf{R} = N^2 - 1 < 2N^2$
- The control is inside H

Simple test case

- Two qubits implemented by $n = 2$ spin-1/2 particles (electron spin + nuclear spin)
- Dimension $N = 2^n = 4$, $\dim_{\mathbf{R}} M(\mathbf{C}, N) = 2N^2 = 32$
- NB. (i) Embedding coordinates on $SU(4)$ ($\dim_{\mathbf{R}} = N^2 - 1 = 15$, but no global chart on the compact group)

  (ii) For spin-1 particles, tensor product with $SU(3)$ instead of $SU(2)$

Simple test case

- Control = microwave pulse on the electron spin only
- After suitable approximations (RWA), Hamiltonian with real controls $u_1(t)$, $u_2(t)$ is

$$H = H_0 + u_1 H_1 + u_2 H_2$$

$$H_0 = \omega_I IZ + A_{\|} ZZ + A_{\perp} ZX, \quad H_1 = \Omega_R XI, \quad H_2 = -\Omega_R YI$$

- Sparse Hermitian matrices ($-iH_k \in \mathfrak{su}(4)$, $k = 0,1,2$, Kronecker product)

$$IZ = I \otimes \sigma_z, \quad ZZ = \sigma_z \otimes \sigma_z, \quad ZX = \sigma_z \otimes \sigma_x, \quad XI = \sigma_x \otimes I, \quad YI = \sigma_x \otimes I$$

$$\sigma_x = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad \sigma_y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}, \quad \sigma_z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

$$ZX = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 \\ 0 & 0 & -1 & 0 \end{bmatrix} \quad \textit{etc.}$$

Simple test case

- Target = quantum gate in $SU(N)$
- Standard geometric control theory: rank condition on the Lie algebra implies (i) controllability when $A_\perp \neq 0$, (ii) only strict orbit reached for $A_\perp = 0$
- Numerical example: $\omega_l = 0$, $A_\perp = 0$, fixed final time and maximisation of *fidelity wrt.* a given target quantum gate Q_f

$$f(Q(t_f), Q_f) \to \max, \quad f(Q_1, Q_2) := |(Q_1|Q_2)|^2/N^2, \quad (Q_1|Q_2) := \mathrm{tr}(Q_1^\dagger Q_2)$$

OptimalControl.jl

- Discretising OCPs...

$$g(x(t_0), x(t_f)) + \int_{t_0}^{t_f} f^0(x(t), u(t))\,\mathrm{d}t \to \min$$

suject to

$$\dot{x}(t) = f(x(t), u(t)), \quad t \in [t_0, t_f]$$

plus boundary and path constraints

$$b(t_0, x(t_0), t_f, x(t_f)) \leq 0$$

$$c(x(t), u(t)) \leq 0, \quad t \in [t_0, t_f]$$

- ... into NLPs: $h_i := t_{i+1} - t_i$,

$$g(X_0, X_N) + \sum_{i=0}^{N} h_i f^0(X_i, U_i) \to \min$$

subject to

$$X_{i+1} - X_i - h_i f(X_i, U_i) = 0, \quad i = 0, \ldots, N-1$$

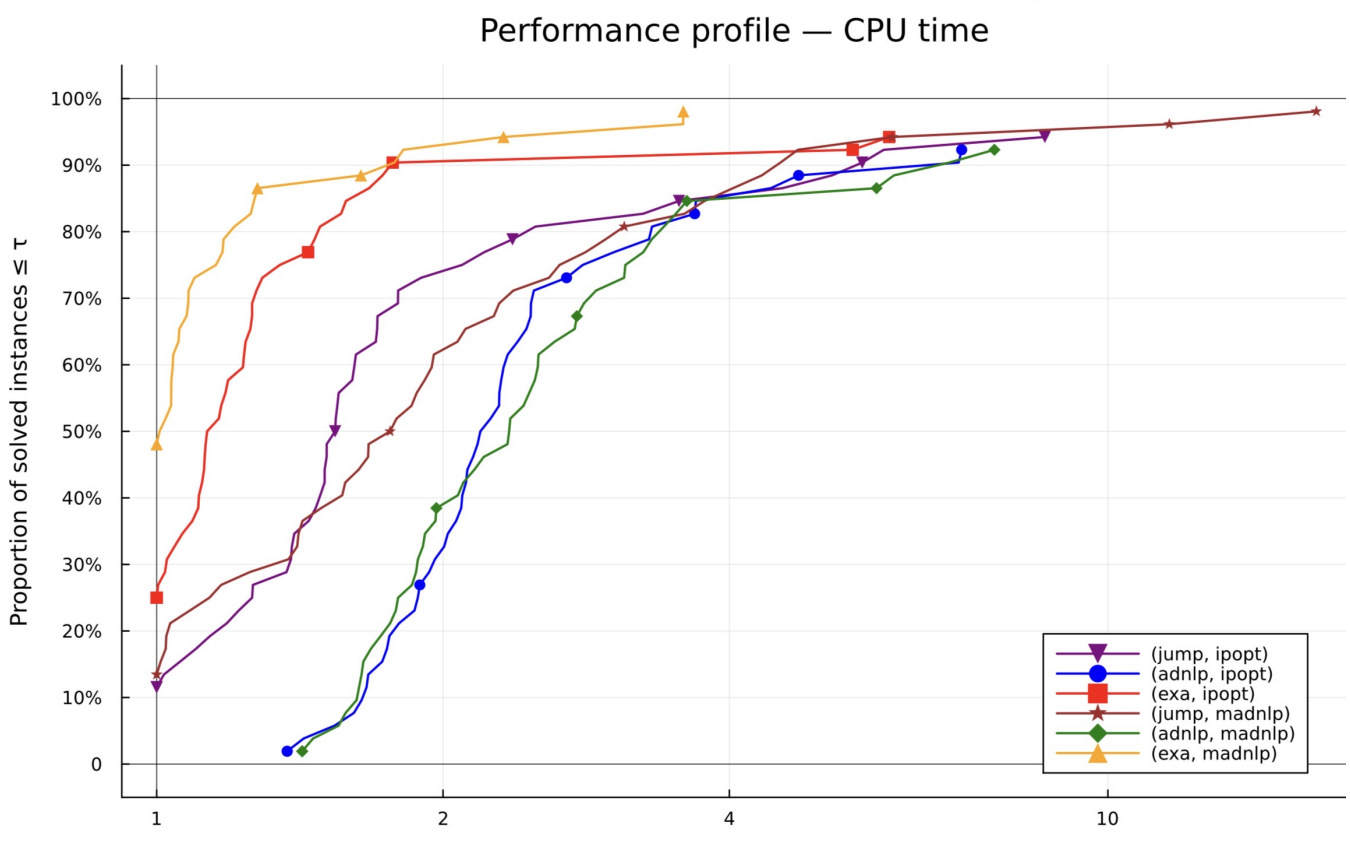
plus other constraints on $X := (X_i)_{i=0,N}$ and $U := (U_i)_{i=0,N}$

$$b(t_0, X_0, t_N, X_N) \leq 0$$

$$c(X_i, U_i) \leq 0, \quad i = 0, \ldots, N$$

OptimalControl.jl

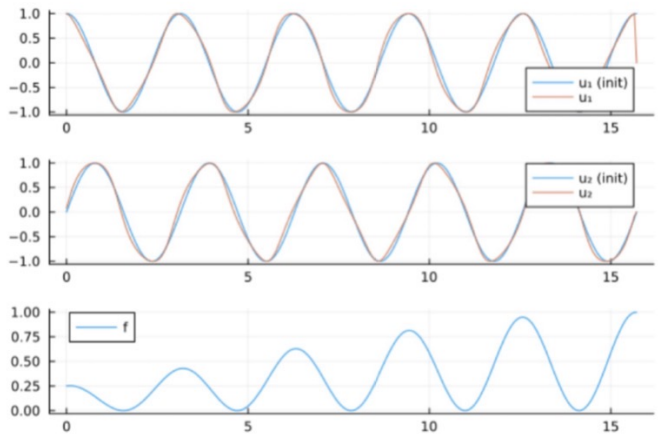
- Dev. team: O. Cots, J. Gergaud, P. Martinon (Univ. Toulouse, CNRS)
- Abstract syntax as close as possible to the math
- Leveraging optimisation modellers and solvers
- ADNLPModels / ExaModels + Ipopt / MadNLP / Knitro / Uno
- Runs on CPU + GPU



Solve 1/2 (direct method)

```
N = 2 ^ 2 # 2 spins 1/2
ZZ = σz ⊗ σz
ZX = σz ⊗ σx
XI = σx ⊗ id
YI = σy ⊗ id
CROT = T[ 0  0 -ı  0
          0  1  0  0
         -ı  0  0  0
          0  0  0  1]
Q0 = @v Matrix{T}(I, N, N)
Ωᵣ = Al / 10
H₀ = @v Al * ZZ
H₁ = @v Ωᵣ * XI
H₂ = @v -Ωᵣ * YI
Qf = @v exp(-ı * tf * Al * ZZ) * CROT
```

```
o = @def begin
    t ∈ [0, tf], time
    q ∈ R^(2N^2), state
    u ∈ R², control
    q(0) == Q0
    q̇(t) == (H₀ + u₁(t) * H₁ + u₂(t) * H₂) ⊙ q(t)
    u₁(t)^2 + u₂(t)^2 ≤ 1
    1 - fid(q(tf), Qf) → min
end
```



Solve 2/2 (shooting)

```
# Regular maximising control from PMP

function ur(q, p)
    s1 = dot(p, H₁ ⊙ q)
    s2 = dot(p, H₂ ⊙ q)
    return [s1, s2] / sqrt(s1^2 + s2^2)
end

fr = Flow(o, ur) # Regular Hamiltonian flow

# Shooting function

t0 = 0
q0 = state(sol)(t0)
p0 = costate(sol)(t0) # Initial guess

function shoot!(s, p0)
    qf, pf = fr(t0, q0, p0, tf)
    s .= pf .- gradient(x -> fid(x, Qf), qf)
    return nothing
end
```

Going further

- More cost functionals, more constraints... and more qubits
- WIP: direct shooting (control discretisation only) + adapted Lie group ODE numerical schemes
- See also

www.nature.com/scientificreports

scientific reports

Check for updates

OPEN

Fast protocols for charging a three-spin-chain quantum battery

Vasileios Evangelakos, Emmanuel Paspalakis & Dionisis Stefanatos✉

Going further

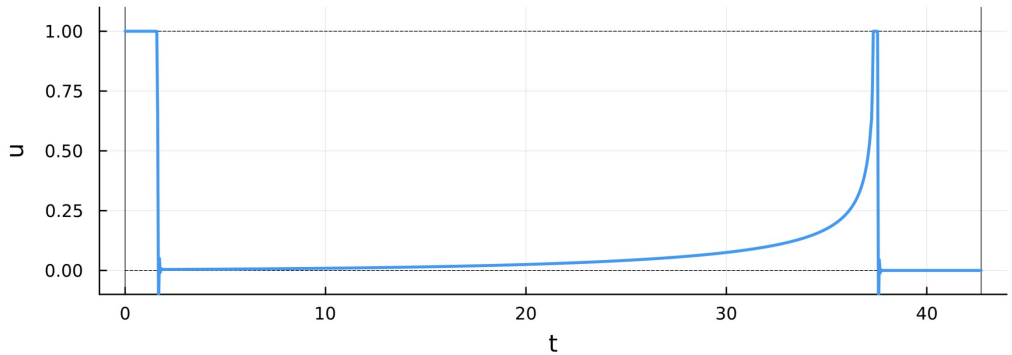
Documentation Tutorials Applications Problems Citing GitHub Contributors

Magnetic Resonance Imaging

Search docs (Ctrl + /)

- Introduction
- Bloch equation
- Saturation problem
  - Time-minimal saturation problem
  - Direct method
  - Indirect method

```
function shoot!(s, pz0, t1, t2, t3, tf, q1, p1, q2, p2, q3, p3)

    p0 = [-1, pz0]

    q1_, p1_ = f1(t0, q0, p0, t1)
    q2_, p2_ = fs(t1, q1, p1, t2)
    q3_, p3_ = f1(t2, q2, p2, t3)
    qf , pf  = f0(t3, q3, p3, tf)

    s[1] = - p1[1] * zs + p1[2] * q1[1]   # H1 = H01 = 0 on the horizontal
    s[2] = q1[2] - zs                     # singular line, z=zs
    s[3] = p3[1]                          # H1 = H01 = 0 on the vertical
    s[4] = q3[1]                          # singular line, y=0
    s[5] = qf[2]                          # z(tf) = 0

    # matching conditions
    s[ 6: 7] = q1 - q1_
    s[ 8: 9] = p1 - p1_
    s[10:11] = q2 - q2_
    s[12:13] = p2 - p2_
    s[14:15] = q3 - q3_
    s[16:17] = p3 - p3_

end
```



- control-toolbox.org